PRÁCTICA 3:
PATRONES DISTRIBUIDOS CON COLAS EN RABBITMQ



Resumen

En esta práctica se implementará un sistema de reparto de pedidos utilizando varios patrones sobre colas con RabbitMQ. Se desarrollarán dos aplicaciones con el framework .Net para representar la creación de pedidos y su reparto, con una comunicación asíncrona sobre colas y simulando el desacoplamiento característico de sistemas distribuidos mediante colas de mensajes.

1. Objetivos

- Comprender los distintos patrones distribuidos aplicables a colas como mecanismo de desacoplamiento en sistemas distribuidos.
- Entender el rol de RabbitMQ como middleware de mensajería y *bróker* intermediario.
- Asimilar el concepto de cola y durabilidad de mensajes.

2. Fundamentos teóricos

RabbitMQ

Imaginemos que necesitamos coordinar la comunicación entre múltiples aplicaciones que pueden estar ejecutándose en diferentes servidores, en diferentes momentos, y a diferentes velocidades. RabbitMQ resuelve este desafío actuando como un bróker de mensajes.

RabbitMQ es un bróker de mensajes, lo que significa que actúa como un intermediario inteligente entre aplicaciones que necesitan intercambiar información. Con este fin, implementa el protocolo **AMQP (*Advanced Message Queuing Protocol*)**, un estándar abierto para mensajería robusta. Como bróker completo, RabbitMQ no solo maneja el almacenamiento temporal de mensajes, sino que también proporciona garantías sobre la entrega, permite el uso de diferentes patrones de enrutamiento, y mantiene la confiabilidad incluso cuando algunas partes del sistema fallan.

Uso de Colas en Sistemas Distribuidos

Las colas son estructuras de datos que típicamente siguen el principio **FIFO (*First In, First Out*)**, aunque en sistemas de mensajería pueden implementar otros esquemas de ordenación según las necesidades.



En el contexto de sistemas distribuidos, las colas actúan como mecanismos fundamentales de **coordinación y desacoplamiento**:

Cuando un productor envía un mensaje a una cola, no necesita saber quién lo va a procesar, cuándo lo van a procesar, o incluso si hay alguien disponible para procesarlo en ese momento. Esta independencia permite que cada componente del sistema evolucione, escale y falle de manera independiente.
Las colas también proporcionan un mecanismo natural de control de flujo. Si los consumidores procesan mensajes más lentamente de lo que los productores los generan, la cola actúa como un ***buffer*** que absorbe las diferencias de velocidad entre ambos extremos. Esto es especialmente valioso en sistemas donde la carga de trabajo puede variar significativamente a lo largo del tiempo.
Otro aspecto crucial es la **tolerancia a fallos**. Las colas pueden configurarse para ser duraderas (***persistent***), lo que significa que los mensajes sobreviven a reinicios del sistema. Además, mecanismos como las confirmaciones o acuses de recibo **(*acknowledgments*, ACKs)** permiten garantizar que los mensajes no se pierdan incluso si un consumidor falla mientras está procesando un mensaje.
Importante: Cuando un consumidor confirma la recepción de un mensaje, dicho mensaje deja de estar disponible para el resto de los consumidores.

- ¿Qué sistemas conocéis que hagan uso de colas?

Callbacks

Un *callback* es una función que se pasa como argumento a otra función para ser ejecutada posteriormente cuando ocurra un evento específico.
En RabbitMQ, los consumidores utilizan *callbacks* para procesar mensajes: en lugar de preguntar constantemente si hay nuevos mensajes, se registra una función que será invocada automáticamente cada vez que llegue uno. Esto permite que el programa reaccione a eventos sin bloquear su ejecución esperando que ocurran.

3. Prerrequisitos

Docker Desktop debe estar instalado en nuestra máquina. Si no es así, podemos seguir los pasos de la Práctica 1 para instalarlo.

RabbitMQ

Haremos uso de un contenedor de RabbitMQ. Para ello, bajaremos la imagen desde *Docker Hub*. En este caso, usaremos una que incluye la web de gestión ya habilitada, desde la que se podrá consultar el estado tanto del bróker como de las colas presentes. Desde una consola ejecutamos:

```
docker run -d --name rabbitmq-ssdd -p 5672:5672 -p 15672:15672 rabbitmq:4.3.6-management
```

Desde un navegador, abrimos http://localhost:15672 (usuario *guest*, contraseña *guest*) y comprobamos las distintas opciones que ofrece la web y que todo funciona correctamente. Se recomienda crear un nuevo usuario desde la web para ser usado durante la práctica.

Entorno de Desarrollo

Similar al utilizado en las prácticas 1 y 2.

4. Desarrollo

Se nos ha encargado la tarea de desarrollar un nuevo sistema de reparto de pedidos para nuestro restaurante. Los pedidos se reciben desde varias fuentes (irrelevantes en esta práctica). Una vez que cada pedido está listo, se ofrece a una serie de repartidores para que uno de ellos se encargue de hacer llegar dicho pedido a los hambrientos e impacientes clientes.

Ejercicio 1: Productor/Consumidor

Para este ejercicio inicial, supondremos que el restaurante sólo tiene un repartidor y éste es el encargado de entregar todos los pedidos. Nos encontramos ante el caso más sencillo en el uso de colas: el **patrón productor/consumidor**. Un productor (el restaurante en nuestro caso) deposita mensajes (pedidos) en una cola (imaginemos una larga mesa en la que los pedidos que se prepararon primero están más cerca de la puerta/repartidor), y un consumidor (el repartidor) los recibe siguiendo el principio FIFO.



Proyecto Productor

El primer paso será crear un proyecto de consola desde *VS Code*, que llamaremos *Restaurante*, en el que incluiremos la lógica necesaria para comunicarnos con nuestra instancia de RabbitMQ y publicar mensajes en una cola. Para evitar que la aplicación utilice instrucciones de nivel superior (*top-level statements*), forzaremos el uso del método *main* al crear el proyecto:

```
dotnet new console --use-program-main -n Restaurante
```

Para interactuar con RabbitMQ desde .Net necesitamos instalar su cliente desde un paquete NuGet. Desde una terminal, navegamos hasta nuestro proyecto y ejecutamos:

```
dotnet add package RabbitMQ.Client --version 7.2.1
```

Necesitamos instalar dos paquetes adicionales para trabajar con los ficheros de configuración que veremos a continuación:

```
dotnet add package Microsoft.Extensions.Configuration
dotnet add package Microsoft.Extensions.Configuration.Json
```

Dentro del fichero Program.cs, añadimos los *using* necesarios:

```
using System.Diagnostics;
using Microsoft.Extensions.Configuration;
```

Y creamos un método añade a la configuración los valores que se encuentren en el fichero de configuración appsettings.json (o appsettings.Development.json si nos encontramos depurando la aplicación):

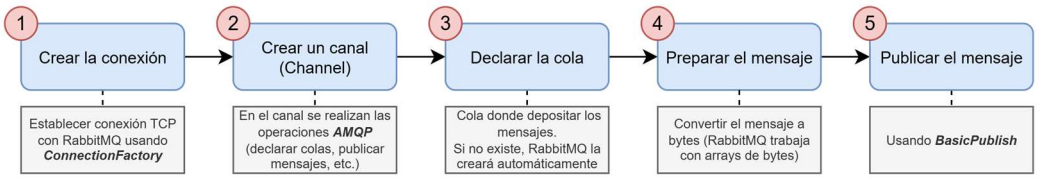
```
private static IConfiguration BuildConfiguration()
    {
        var configurationBuilder = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false);

        if (Debugger.IsAttached)
        {
            configurationBuilder.AddJsonFile("appsettings.Development.json", optional: true);
        }
        return configurationBuilder.Build();
    }
```

Crearemos una clase *Producer* que recogerá la lógica de envío de mensajes, e incluimos las referencias mínimas necesarias:

```
using System;
using System.Text;
using RabbitMQ.Client;
```

Ahora debemos preparar al productor (clase *Producer*) para que pueda interaccionar con el bróker. Los pasos a seguir son los siguientes:



1 Declaramos las variables de clase necesarias para la comunicación con RabbitMQ en el fichero de configuración appsettings.json. Deberéis crear también un appsettings.Development.json con, de momento, el mismo contenido:

```
{
  "RabbitMQ": {
    "HostName": "localhost",
    "Port": 5672,
    "UserName": "guest",
    "Password": "guest",
    "QueueName": "pedidos"
  }
}
```

En la clase *Producer.cs*, creamos las siguientes variables:

```
 private readonly IConfiguration _configuration;
 private readonly string _queueName;
 private IChannel? _channel;
```

El constructor recibe por parámetro el objeto de configuración preparado en *Program.cs* e inicializa el nombre de la cola:

```
public Producer(IConfiguration configuration)
{
    _configuration = configuration;
    _queueName = _configuration["RabbitMQ:QueueName"]; // Se obtiene el valor desde appsettings.json
}
```

Ahora, necesitamos el método *InitializeConnection()* que, de forma asíncrona, inicialice la conexión a RabbitMQ utilizando la factoría de conexiones. Primero recuperamos el puerto:

```
int port =  int.TryParse(_configuration["RabbitMQ:Port"], out int parsedPort) ? parsedPort : 5672;
```

Después, creamos la conexión desde una factoría de conexiones:

```
var factory = new ConnectionFactory
{
    HostName = _configuration["RabbitMQ:HostName"] ?? "localhost", // ?? operador de coalescencia
    Port = port,
    UserName = _configuration["RabbitMQ:UserName"] ?? "guest",
    Password = _configuration["RabbitMQ:Password"] ?? "guest"
};

var connection = await factory.CreateConnectionAsync();
```

2 Sin salir del método, creamos el canal:

```
_channel = await connection.CreateChannelAsync();
```

3 Y declaramos la cola:

```
await _channel.QueueDeclareAsync(
    queue: _queueName,
    durable: true, // La cola sobrevive a reinicios del servidor
    exclusive: false, // La cola puede ser usada por múltiples conexiones
    autoDelete: false, // La cola no se elimina automáticamente
    arguments: null
);
```

Dentro de *Producer*, creamos un método asíncrono *SendMessage* que acepta un parámetro *pedido* de tipo *string* y devuelve un booleano que indicará el correcto envío (o no) del mensaje a la cola. En este método:

4 Preparamos el mensaje:

```
var body = Encoding.UTF8.GetBytes(pedido);
```

5 Y, finalmente, lo enviamos a la cola:

```
await _channel.BasicPublishAsync(
    exchange: string.Empty, // Exchange por defecto. Se usará en P4
    routingKey: _queueName, // Nombre de la cola como routing key
    body: body
);
```

Se deberán controlar las posibles excepciones/valores nulos en el envío y devolver *false* en caso de producirse alguna.

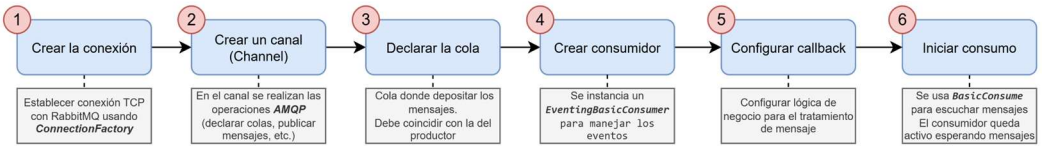
Ahora solo queda, desde *main*, llamar a *BuildConfiguration*:

```
var configuration = BuildConfiguration();
```

Finalmente, instanciaremos *Producer*, inicializaremos su conexión, y solicitaremos **de forma recurrente** al usuario un nombre de pedido por consola e invocaremos a *SendMessage*. Se deberá mostrar un menaje por consola indicando el éxito o fracaso en el envío del mensaje a la cola.

Proyecto Consumidor

De forma similar al proyecto *Restaurante*, creamos un proyecto de consola llamado *Repartidor* y seguimos unos pasos muy parecidos al proyecto anterior:



En este caso, creamos una clase *Consumer* y reutilizamos las configuraciones, variables y la lógica del constructor de la clase *Producer*.
Los pasos nuevos o que requieren cambios respecto al productor son los siguientes:

4 En un nuevo método asíncrono llamado *StartConsuming* creamos el consumidor:

```
var consumer = new AsyncEventingBasicConsumer(_channel);
```

5 Y configuramos el *callback* que se ejecutará automáticamente cada vez que se reciba un mensaje:

```
consumer.Received += async(model, ea) =>
{
    var body = ea.Body.ToArray(); //contenido del mensaje (array de bytes)
    var message = Encoding.UTF8.GetString(body); //se convierte de vuelta a string

    Console.WriteLine($"Pedido recibido: {message}");

    await ProcessOrder(message); // Aquí va la lógica de procesamiento del mensaje
};
```

Donde ***model*** es una referencia al objeto que desencadenó el evento y ***ea*** son los argumentos del evento (**E**vent **A**rguments); ***+=*** crea una suscripción al evento received de consumer y ***=>*** referencia al *callback* (el bloque entre llaves, en este caso).
El método *ProcessOrder* deberá incluir la lógica necesaria para que el repartidor espere, de manera aleatoria, entre 1 y 5 segundos cada vez que se atiende un pedido.

- ¿Quién es en este caso 'model'? ¿De qué tipo es?

6 Una vez configurado el consumidor, podemos iniciarlo (dentro de *StartConsuming*):

```
_channel.BasicConsumeAsync(
    queue: _queueName,
    autoAck: true, // Confirmación automática de recepción del mensaje
    consumer: consumer
);
```

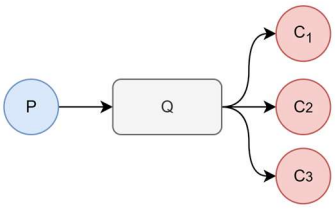
En el método *main*, simplemente quedará instanciar *Consumer*, llamar a *StartConsuming,* indicar al usuario que el repartidor está listo y esperar la pulsación de una tecla para terminar la ejecución del programa.

Nota: Es muy recomendable cerrar la conexión y el canal una vez no se necesiten (dispose).

Ejercicio 2: Colas de trabajo y confirmaciones de recepción (ACKs)

Al restaurante le ha ido tan bien con el nuevo sistema de reparto que ha decidido contratar a más repartidores para poder atender la demanda. Debido a la expansión, el restaurante ha considerado mejorar la información que se gestiona de cada pedido. Ahora, cada Pedido incluirá: *Id, Cliente, Nombre del producto y Distancia*.
Por su parte, **los repartidores podrán decidir si atienden o no un pedido dependiendo de la distancia del destino y devolverlo a la cola**.

En este ejercicio usaremos el patrón **colas de trabajo *(work queues)*** como extensión del patrón del productor/consumidor. En este patrón, varios consumidores se conectan a una misma cola, y RabbitMQ distribuye automáticamente los mensajes entre ellos usando *round-robin* por defecto. Este mecanismo permite procesar mensajes más rápidamente al paralelizar el trabajo entre múltiples trabajadores, o distribuir tareas complejas dividiéndolas en subtareas más pequeñas.



Al permitir que los repartidores rechacen mensajes, se introducen dos problemas significativos:

1. **Ruptura del orden FIFO**: Los mensajes rechazados se vuelven a encolar, lo que rompe el orden original en que fueron enviados.
2. **Riesgo de inanición:** Si todos los repartidores rechazan sistemáticamente ciertos pedidos (por ejemplo, los más lejanos), estos mensajes podrían no procesarse nunca, quedando atrapados en un ciclo de rechazos.

Nota: En este ejercicio simplificado asumiremos que siempre habrá algún repartidor dispuesto a aceptar cualquier pedido, pero en sistemas reales esto requeriría mecanismos adicionales como colas de dead letter o límites de reintentos.

Debemos adaptar los proyectos del *Ejercicio 1* para incorporar los cambios solicitados.

Pedido

Los mensajes que se depositarán (y extraerán) de la cola pasan a ser objetos. Se ha de crear una nueva clase con los atributos necesarios. En el productor, se crearán instancias de la clase *Pedido*. Para poder enviarse a la cola, primero convertiremos el objeto a *json* para después enviarlo como array de bytes.

```
var pedidoJson = JsonConvert.SerializeObject(pedido);
var body = Encoding.UTF8.GetBytes(pedidoJson);
```

El consumidor ha de realizar la operación inversa una vez obtenga los bytes del mensaje.

ACKs

Para implementar la capacidad de los repartidores de rechazar un pedido, configuraremos el consumidor para cambiar la notificación de recepción de mensajes a modo manual:

```
channel.BasicConsumeAsync(
    queue: _queueName,
    autoAck: false, // CAMBIO: Desactivar ACK automático para control manual
    consumer: consumer
);
```

Previa a la llamada a *ProcessOrder*, comprobaremos si el repartidor acepta o no el pedido invocando a un nuevo método *AcceptOrder* que tomará la distancia como parámetro. Este método devolverá *true* o *false* dependiendo de la decisión tomada. El mecanismo para aceptar o rechazar el pedido puede hacerse mediante una probabilidad preestablecida (rechazar el 20% de los pedidos) o ponderando la tolerancia al rechazo en base a la distancia del pedido (a mayor distancia, mayor probabilidad de rechazo).
Si se acepta el pedido, se comunicará al bróker la recepción y se llamará a procesar pedido:

```
if(AcceptOrder(pedido.Distancia))
{
    Console.WriteLine($"[(·)] Pedido {pedido.Id} para {pedido.Cliente} aceptado.");
    channel.BasicAckAsync(ea.DeliveryTag, false);
    await ProcessOrder(pedido);
}
```

En caso contrario, devolvemos el mensaje a la cola:

```
else
{
    Console.WriteLine($"[X] Pedido {pedido.Id} para {pedido.Cliente} rechazado. \n");
    channel.BasicNackAsync(ea.DeliveryTag, false, true);
    return;
}
```

-¿Qué finalidad tienen los parámetros de BasicAck y BasicNack?

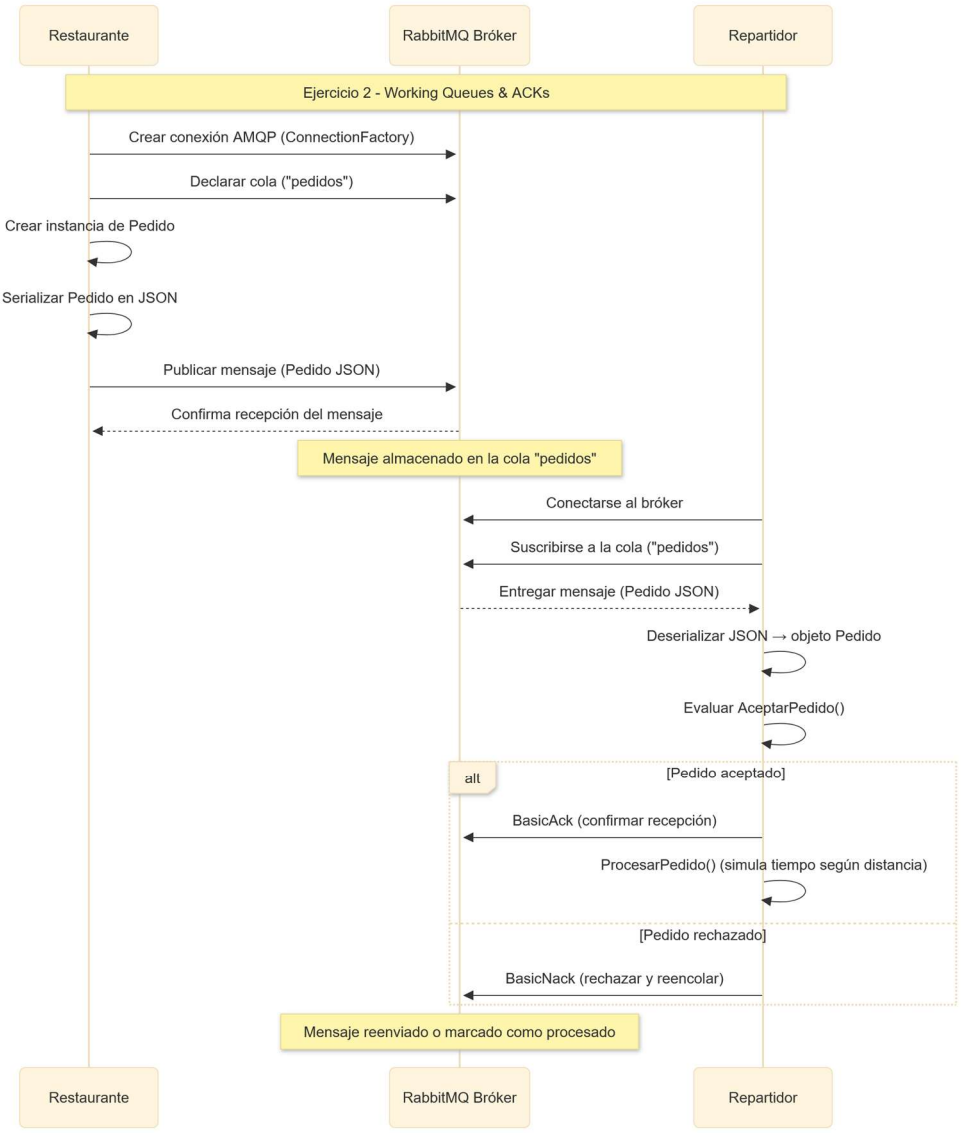
Haremos, también, que *ProcessOrder* espere un tiempo, ya no aleatorio sino variable, dependiendo de la distancia del pedido (p.ej. 1km = 1”). Recordemos que los métodos de pausado del proceso en .Net trabajan en milisegundos, y su implementación varía dependiendo de si el método es síncrono o asíncrono:

```
 // En método síncrono:
Thread.Sleep(pedido.Distancia * 1000); // esperará el valor de distancia en segundos

// En método asíncrono (usando async/await):
await Task.Delay(pedido.Distancia * 1000); // esperará el valor de distancia en segundos
```

Vista general del proceso distribuido

El siguiente diagrama de secuencia representa las interacciones entre las tres aplicaciones que conforman el ejercicio durante el proceso de creación, envío, recepción y procesamiento de un mensaje.



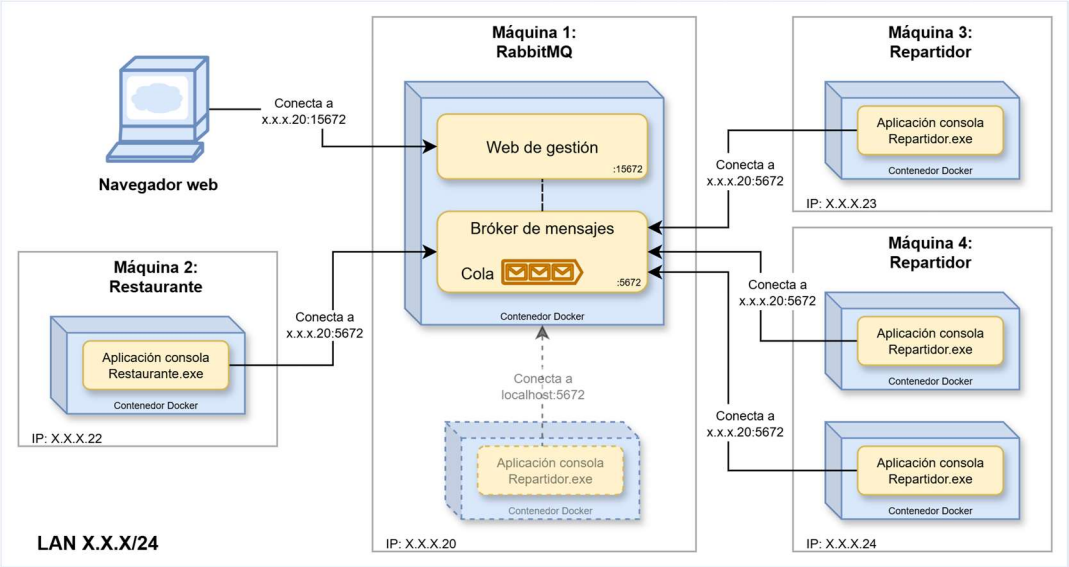
Despliegue

Una vez que tanto el productor como el consumidor están desarrollados y se ha probado su funcionamiento en el entorno local, debemos crear sus imágenes y lanzar ambas aplicaciones en contenedores Docker.

Cada grupo deberá desplegar en su instancia de Docker una de las partes del sistema (Productor, Consumidor/es o Bróker) y probar que el sistema distribuido funciona entre las tres partes.

El escenario final deberá parecerse al de la siguiente figura. Pueden existir variaciones dependiendo del numero de alumnos en el grupo o de la cantidad final de repartidores desplegados. Una máquina no tiene por qué trabajar en exclusiva

con un único rol (p.ej. Máquina1 puede albergar el bróker y uno o varios repartidores):



Nota: Aseguraos de abrir los puertos necesarios en vuestra máquina y de formar parte de la misma red local en el momento de las pruebas.

5. Preguntas de Autoevaluación

1. ¿Qué pasa si el productor se desconecta antes de que el consumidor reciba el mensaje?
2. ¿Qué pasa si uno de los consumidores detiene su ejecución? ¿Y si se reconecta tiempo después?
3. ¿Qué les ocurre a los mensajes de una cola si el contenedor de RabbitMQ se (a) pausa, (b) detiene, (c) se borra?
4. ¿Qué pasa si la recepción de mensajes está en modo manual y un consumidor no confirma o rechaza el mensaje recibido?
5. ¿Cuál es la diferencia entre una cola duradera (durable) y un mensaje persistente? ¿Es necesario configurar ambas para garantizar que los mensajes sobrevivan a un reinicio del bróker?
6. ¿Qué sucede si un productor envía un mensaje a una cola que no existe?